



AJAY KADIYALA - Data Engineer

Follow me Here:

LinkedIn:

<https://www.linkedin.com/in/ajay026/>

Data Geeks Community:

<https://lnkd.in/geknpM4i>

Data Engineering **Take-Home** & Practical Guide (Templates + Rubrics + Real Tasks)

Table of Contents:

1. How to use this pack
2. What interviewers actually grade
3. Timeboxing strategy (so you finish strong)
4. Standard submission structure
5. Core patterns you'll reuse in every practical round
6. The Take-Home Problem Bank (realistic task styles)
7. Edge cases checklist
8. Testing pack
9. Data quality + validation framework
10. Performance + cost checklist
11. Observability basics for take-homes
12. Documentation that reviewers love
13. CI/CD + packaging
14. Security & governance
15. Reviewer red flags
16. Final submission checklist
17. 10 Real Practical Interview Scenarios (Questions + Personalized Answers)

0.1 Who this pack is for

This pack is for you if you:

- get nervous in practical rounds because you don't know **what to write first**
- solve the task, but your submission still feels “not professional”
- miss **edge cases, rerun safety, tests, or README clarity**
- waste time overthinking architecture

If you already do take-homes regularly, use this pack for:

- faster structure
- better review checklist
- stronger “production mindset” write-up

0.2 Before you start: pick your time budget

In take-homes, time is the real boss. Choose one:

A) 2 hours (fast round / live practical)

- Goal: Correct output + clean structure + basic README
- Skip: fancy frameworks, docker, extra optimization

B) 4 hours (standard take-home)

- Goal: Correctness + edge cases + 2–4 tests + clear tradeoffs
- Add: validations + rerun/idempotency explanation

C) 8 hours (weekend take-home)

- Goal: all of above + stronger docs + better test coverage + performance notes

0.3 The 5-step workflow (my teaching method)

Whenever you do any task in this pack, follow this flow:

Step 1: Restate the problem in your words (2–3 lines)

- What is input?
- What is output?
- What are the rules?

Step 2: Write assumptions (this saves you in review)

Examples:

- “Input can have duplicates”
- “Timestamps can be late”
- “Nulls can exist in key fields”
- “We need rerun-safe loads”

Step 3: Decide the simplest approach

Pick only what’s needed:

- Batch ETL? SQL? dbt? Streaming concept?
- Don’t mix everything unless asked.

Step 4: Build in this order

1. skeleton repo + README draft
2. core logic (get output correct)
3. validations + edge cases
4. tests
5. final notes (tradeoffs + improvements)

Step 5: Self-review with the rubric

Before submission, score yourself:

- correctness 
- clarity 
- tests 
- rerun story 
- edge cases 
- tradeoffs 

This is exactly what the interviewer is doing anyway.

0.4 How to navigate this pack (don't read everything)

Use it based on your situation:

If you have an interview in 24–48 hours

- Section 1 (grading rubric)
- Section 2 (timeboxing)
- Section 3 (repo structure)
- Section 6 (edge cases)
- Section 15 (final checklist)

If you want to become strong in practical rounds (2–3 weeks)

- Do 3 problems from Section 5:
 - 1 Batch task

- 1 SQL analytics task
- 1 dbt / data modeling task
- After each problem, use Section 7 + Section 11 templates

If you are targeting product-based companies

- Focus more on:
 - idempotency + reruns
 - testing
 - clean assumptions + tradeoffs
 - performance thinking (not overengineering)

0.5 What “good submission” looks like (simple checklist)

A good submission usually has:

- a clean folder structure
- code that runs without manual edits
- clear README (setup + run steps + assumptions)
- handles common bad data cases
- 2–4 meaningful tests (not fake tests)
- short “tradeoffs + next improvements” section

A great submission adds:

- incremental load logic (or at least explains how)
- data validation strategy (what happens on failure)
- performance notes (what would break at scale)

0.6 My rule: don’t overbuild

Most candidates lose because they try to impress with complexity.

So follow this rule:

“Simple + correct + clear beats complex + incomplete.”

If you’re confused between two approaches:

- pick the one you can **explain confidently**
- pick the one you can **test quickly**
- pick the one that is **rerun safe**

0.7 Quick warm-up exercise (do this before Section 5)

Take any sample dataset (CSV/JSON). In 10 minutes, write:

1. 3 assumptions
2. 3 edge cases
3. 3 validations
4. 2 tests you would write
5. 2 tradeoffs you would mention

This trains your brain to think like a reviewer.

1.1 The Real Scoring Breakdown

Think of your submission being scored out of 100:

Area	Weight (Typical)	What they check
Correctness	30–40%	Does it solve the problem fully?
Edge Case Handling	15–20%	Did you think beyond happy path?
Code Clarity & Structure	15–20%	Is it readable and clean?
Testing & Validation	10–15%	Did you verify your logic?
Production Thinking	10–15%	Is it rerun safe? Scalable?
Documentation	5–10%	Is README clear and honest?

Notice something?

Correctness is NOT 100%.

That's where most people lose.

1.2 What “Correct but Rejected” Looks Like

Let me show you common rejection patterns:

✗ Case 1: Output correct, but...

- No handling of duplicates
- No null handling
- No rerun safety
- No explanation of assumptions

Reviewer thinks:

“Works on small test. Won't survive production.”

✗ Case 2: Overengineered

- Docker
- 10 configs
- Over-complicated class hierarchy
- No actual edge-case handling

Reviewer thinks:

“Spent time impressing, not solving.”

✗ Case 3: No clarity

- No README
- No assumptions
- No explanation of decisions

- No mention of tradeoffs

Reviewer thinks:

“Hard to review. Risky hire.”

1.3 What a Strong Submission Feels Like

When reviewers open a strong submission, they feel:

- I understand the structure in 30 seconds.
- I can run this easily.
- The candidate thought about bad data.
- The candidate thought about reruns.
- The candidate knows real-world pipelines.
- The candidate is honest about limitations.

That “feeling” matters a lot.

1.4 What Interviewers Secretly Love

There are 5 things that instantly increase your score:

1 Clear Assumptions Section

Example:

- “Assume input may contain duplicates.”
- “Assume timestamps may arrive late.”
- “Assume reruns are possible.”

This shows maturity.

2 Idempotency Mention

Even if you don’t fully implement it, saying:

“This pipeline is designed to be rerun safe by using upsert logic...”

This signals production mindset.

3 Tradeoff Section

At the end of README, add:

- Why I chose this approach
- What I simplified
- What I would improve with more time

Interviewers LOVE this.

4 Simple but Meaningful Tests

Not 20 fake tests.

Just:

- duplicate case
- null case
- edge case

That's enough.

5 Edge Case Checklist Covered

Even if problem doesn't mention it:

- empty file?
- invalid schema?
- missing columns?
- duplicate keys?

Thinking about these separates average from strong engineers.

1.5 The Reviewer Mindset (Think Like Them)

When reviewing, they ask:

- Would I trust this person in production?
- Would this person break pipelines?
- Would this person ask good questions?
- Can this person debug issues?
- Does this person think beyond toy data?

Your job is to answer these silently through your submission.

1.6 Mini Exercise (Do This Now)

Take any past take-home you did.

Ask yourself:

- Did I write assumptions clearly?
- Did I mention rerun safety?
- Did I handle duplicates?
- Did I document tradeoffs?
- Did I add even 2 tests?

If 2 or more are missing — that's why rejections happen.

1.7 Your New Rule

From now on:

Solve the problem

Then solve the production version of the problem

That's how you move from "candidate" to "engineer".

2.1 The Golden Rule (works for every level)

Never code first. Always plan first.

Your first 10–15 minutes decide your result.

Your first 10 minutes must produce:

- 3–5 assumptions
- 3 edge cases
- approach outline (2–3 steps)
- repo structure decision

If you do this, you'll look senior even as a fresher.

2.2 The “Pro Submission” Order (don't change this order)

Build in this order:

- 1. README skeleton (5 mins)**
 - Problem restatement
 - Run steps placeholder
 - Assumptions placeholder
 - Tradeoffs placeholder
- 2. Happy path output (core logic)**
- 3. Edge cases + validations**
- 4. Tests (2–4 meaningful)**
- 5. Final polish (README + final notes)**

This ensures even if time ends, you still have a reviewable submission.

2.3 2-Hour Plan (Live Practical / Quick Round)

0–10 mins: Understand + Write

- restate problem
- assumptions
- edge cases list

- choose approach

10–60 mins: Core logic (happy path)

- get correct output for sample
- keep code simple

60–85 mins: Edge cases + validations

- handle duplicates/nulls/bad types
- add basic logging/errors

85–105 mins: Tests (minimum 2)

- one normal case
- one edge case (duplicate/null/empty)

105–120 mins: README + final notes

- how to run
- assumptions
- tradeoffs
- “if more time”

✓ Deliverable expectation (2 hours):

- correct output
- clean structure
- 2 tests
- decent README

2.4 4-Hour Plan (Most Common Take-Home)

0–15 mins: Plan like a senior

- same as above, but slightly better assumptions
- define input/output schema clearly

15–120 mins: Core logic (happy + complexity)

- core transforms
- incremental logic if possible (or explain it)

120–170 mins: Quality + rerun thinking

- validations
- idempotency approach (even if conceptual)
- failure strategy (quarantine / skip / stop)

170–210 mins: Tests (3–5)

- normal flow
- duplicates
- null key field
- empty input OR schema mismatch

210–240 mins: Documentation + clarity

- README cleaned
- decision log
- tradeoffs and improvements
- performance notes (basic)

✅ Deliverable expectation (4 hours):

- correct + robust
- rerun story
- 3–5 tests
- strong README

2.5 8-Hour / Weekend Plan (For Product Companies / Strong Impression)

Phase 1 (0–30 mins): Design

- dataset profiling quick scan
- schema decisions

- config pattern
- folder structure

Phase 2 (0.5–4 hours): Build properly

- modular code
- validations
- incremental or merge logic where possible

Phase 3 (4–6 hours): Tests + quality

- more test coverage
- golden dataset tests
- negative tests (bad schema)

Phase 4 (6–8 hours): Performance + polish

- performance checklist
- README + design doc
- clear “future work” section

✓ Deliverable expectation (8 hours):

- looks like production-quality mini project

2.6 “What to Cut First” When You’re Running Out of Time

This saves you.

Cut in this order:

1. Docker
2. Fancy architecture patterns
3. Extra optimizations
4. Extra features not asked

Never cut:

- correctness

- README clarity
- basic tests
- edge cases

2.7 Level-Based Expectations (so the pack works for everyone)

For 0–3 yrs

You win by showing:

- clean structure
- correct solution
- basic tests
- honest assumptions
- no overengineering

For 3–8 yrs

You win by showing:

- idempotency/rerun thinking
- validations + failure handling
- performance awareness
- better tradeoffs

For Seniors

You win by showing:

- crisp architecture reasoning
- scaling + cost thinking
- governance/security awareness
- clear prioritization

This pack will guide all 3 levels you just choose how deep you go.

2.8 Quick Practice Drill (5 minutes)

Before every take-home, do this:

Write on paper:

- **Input:**
- **Output:**
- **Assumptions (3):**
- **Edge cases (3):**
- **Validations (3):**
- **Tests (2):**
- **Tradeoffs (2):**

3.1 The default folder layout (use this as-is)

```
project/
  README.md
  requirements.txt      (or pyproject.toml)
  config/
    config.yml          (env settings, paths, flags)
  src/
    main.py             (entry point)
  pipelines/
    pipeline.py          (orchestration / steps order)
  transforms/
    clean.py
    enrich.py
    aggregate.py
  io/
```

```
read.py
write.py
utils/
  logger.py
  common.py
tests/
  test_transforms.py
  test_edge_cases.py
data/          (optional for sample only)
  sample_input.csv
  expected_output.csv
docs/          (optional but strong)
  design.md     (1–2 pages max)
```

Why this layout wins

- Reviewer finds **how to run** fast
- Code is not one messy file
- Tests are clearly separated
- Config is not hardcoded in logic

3.2 The 3 files you must have (minimum standard)

1) README.md

Must contain:

- problem restatement (2–3 lines)
- how to run (copy-paste commands)
- assumptions + edge cases
- output description

- tradeoffs + improvements

2) src/main.py

- single entry point
- calls pipeline steps in order
- no business logic inside main

3) tests/...

Even 2 tests is enough if meaningful.

3.3 The “Don’t Hardcode” rule (very important)

Hardcoding kills scores.

Avoid hardcoding:

- file paths
- column names (when possible)
- dates
- environment settings

Instead:

- put them in config/config.yml
- read config once in main.py
- pass config into pipeline

Even basic config usage signals maturity.

3.4 Entry point flow (what it should look like conceptually)

main.py

- load config
- init logger
- call pipeline.run(config)

pipeline.py

- read input
- validate schema
- apply transformations
- write output
- print summary metrics

That's it. Simple and clean.

3.5 Where SQL/dbt fits in this structure

If the take-home is SQL-heavy

Use:

```
sql/  
  01_staging.sql  
  02_transform.sql  
  03_marts.sql  
README.md
```

Add in README:

- “Run order”
- “Assumptions”
- “Indexes/perf notes”

If the take-home is dbt

Use:

```
dbt_project/  
  models/  
  tests/  
README.md
```

docs/design.md

And still keep your README + design notes.

3.6 What reviewers love seeing in structure (small but high impact)

- ✓ expected_output.csv for sample input
- ✓ design.md with 5 headings:
 - Problem
 - Approach
 - Data assumptions
 - Failure handling
 - Tradeoffs
- ✓ small data/sample_input.csv (only if allowed)

These make review easy.

3.7 The “1-minute reviewer experience” test

Open your own repo like an interviewer:

Ask:

- can I run it in 60 seconds?
- do I understand assumptions in 30 seconds?
- do I see tests immediately?

If yes, you're good.

3.8 Mini exercise (quick)

Pick any take-home you did before.

4.1 Pattern 1 Incremental Load Thinking (even if not asked)

Interviewers love when you think:

“How will this run daily?”

The 3 ways incremental usually works

A) Append-only (events/logs)

- new data comes with new timestamps/IDs
- you load only “new since last run”

B) Upsert (dimension/customer/order tables)

- same keys can come again with updates
- you merge based on primary key + updated_at

C) Rebuild (small tables)

- easiest if table is small
- truncate + reload (but justify cost)

✓ What to write in README (simple):

- “Designed for reruns by using deterministic keys + merge/upsert strategy”
- “If data is append-only, use watermark date filter”

4.2 Pattern 2 Idempotency (Rerun Safety)

Idempotency means:

“If I run it twice, result should not double-count or corrupt.”

Quick ways to achieve idempotency

- write output to a temp path, then swap/overwrite final
- use merge/upsert with unique key
- dedupe input based on keys before aggregations
- partition overwrite for a single day partition

✓ Reviewer signal line:

- “Pipeline is rerun-safe because output is overwritten per partition/day OR merged by key.”

4.3 Pattern 3 Dedup (Most Common Hidden Test)

Duplicates come in 3 forms:

1) Exact duplicate rows

- same full row repeated

2) Duplicate keys with different values

- same customer_id, different address (late update)

3) Near duplicates

- same order, small column differences

Simple dedup decision tree

- If exact duplicates → drop duplicates across all columns
- If key duplicates → keep latest by updated_at (or rule-based survivorship)

✅ Always mention:

- “Dedup rule used: keep latest record per key using timestamp.”

4.4 Pattern 4 Late Arriving Data (Even in Batch)

Late data means:

- yesterday’s partition gets new records today
- or updates come later

Practical handling options

- keep a “lookback window” (e.g., last 3 days) and reprocess
- merge/upsert instead of append
- backfill mode flag in config

✅ Strong README line:

- “Supports late arrivals via configurable lookback window.”

4.5 Pattern 5 Backfill Strategy (Interview favorite)

Backfill = rerunning historical dates.

A simple backfill design

- config takes start_date, end_date
- loop through each day partition
- process + overwrite that day

What interviewers look for:

- you don't overwrite everything blindly
- you can re-run for selected days

4.6 Pattern 6 Failure Handling (What happens when bad data comes?)

Bad data is guaranteed in real life.

The 3 common strategies

A) Fail-fast

- stop pipeline when critical issues happen (missing columns, null keys)

B) Quarantine

- write bad rows into rejects/ with reason column

C) Soft fail

- log warning and continue (only for non-critical)

✅ Include in docs:

- “Critical failures stop job. Non-critical issues are quarantined with reasons.”

4.7 Pattern 7 Schema Evolution (realistic, simple)

Schema changes are common.

Minimum you should cover:

- extra columns appear → ignore or pass-through
- expected column missing → fail-fast
- type mismatch → cast or quarantine

✅ One-liner:

- “Schema validated at read time; missing required columns fail; type issues are quarantined/cast.”

4.8 Pattern 8 Data Contracts (lightweight, interview-winning)

A data contract is simply:

- expected schema
- required fields
- freshness expectation
- uniqueness expectation

Even if you don't use a tool, write it in README/design.md.

4.9 Mini Practice (quick, 2 mins)

Imagine a take-home says:

“Transform daily orders CSV into aggregated revenue by day.”

5.0 How you will use this bank (non-negotiable)

For every lesson, you will submit **5 deliverables**:

1. **README.md** (run steps + assumptions + tradeoffs)
2. **Working solution** (code/SQL/dbt)
3. **Validations** (what you check + what you do on failure)
4. **Tests** (minimum 2, maximum 6)

5. **Final summary** (what you would improve with 2 more hours)

And you will follow the same build order:

README skeleton → happy path → validations → tests → final notes.

Beginner Track (Lesson 1 to 5)

These teach you structure + correctness + confidence.

5.1 Lesson 1 Batch ETL: Clean Orders File

Scenario: You receive a daily orders.csv. It is messy.

Input columns (example):

order_id, customer_id, order_ts, amount, currency, status

Output:

clean_orders written as parquet (or CSV), partitioned by order_date.

What you will implement

- Parse order_ts safely and derive order_date
- Convert amount to numeric
- Remove bad records with missing order_id
- Standardize status values (example: lowercase + trim)

Edge cases you must handle

- order_id is null or empty
- amount is negative or non-numeric
- duplicate order_id appears in the same file
- order_ts is invalid format

Validations you must add

- order_id not null
- amount ≥ 0
- order_ts parse success rate tracked

Tests you must write (minimum 2)

- duplicate order_id keeps the latest record
- null order_id goes to rejects (or fails fast)

What you will write in README

- Your dedup rule (keep latest by timestamp)
- Your failure rule (quarantine vs stop)

5.2 Lesson 2 SQL Metrics: Daily Revenue + Failure Rate

Scenario: You are given a transactions table.

Table:

transactions(user_id, txn_id, txn_ts, amount, status)

Tasks:

- daily revenue (successful only)
- daily failure % (failed / total)
- top 5 users by monthly spend

What you will practice

- GROUP BY + DATE extraction
- window functions for ranking
- correct filtering on status

Edge cases

- null amount
- status contains weird values (Success, success)

Deliverable

- sql/01_metrics.sql
- README with assumptions + indexes/performance note

5.3 Lesson 3 File Merge: Big CSV That Cannot Fit Memory

Scenario: You are given a huge CSV that cannot load fully into memory.

Task:

- read in chunks
- compute total revenue per day
- write incremental results

What you will implement

- chunk processing (pandas chunks or streaming reads)
- incremental aggregation
- safe output write

What you will prove in README

- why your approach is memory safe
- time complexity in simple words

5.4 Lesson 4 Dedup Rules: Survivorship Decision

Scenario: Same customer_id appears multiple times with different values.

Input: customer_id, email, phone, updated_at

Output: dim_customer_latest

You will implement one survivorship rule

- keep latest by updated_at

Must include

- tie breaker rule if timestamps are same
- explanation why this rule is safe

5.5 Lesson 5 Data Quality Basics: Build a Rejects Table

Scenario: You are not allowed to drop bad rows silently.

Task:

- split output into:
 - clean_records
 - reject_records with reject_reason

Must include reject reasons

- missing key
- invalid datatype
- negative amount

Intermediate Track (Lesson 6 to 11)

These teach you production mindset: reruns, incremental, CDC, modeling.

5.6 Lesson 6 Incremental Loads: Process Only New Data

Scenario: You get daily partitions for orders.

Task:

- process only new partitions
- store a checkpoint (last processed date)

You will implement

- a checkpoint file/table
- “rerun the same date” behavior (should not duplicate)

5.7 Lesson 7 Backfill Mode: Process Date Range

Scenario: Data was wrong for last week.

Task:

- config supports: start_date, end_date
- you process each day in range

- you overwrite that day output safely

Must include

- clear CLI or config usage in README
- proof that rerun does not double count

5.8 Lesson 8 CDC Lite: Upserts into Fact Table

Scenario: Same order_id arrives again with updated status or amount.

Task:

- maintain latest row per order_id
- implement upsert logic (merge concept)

What you will show

- how you pick latest version (updated_at)
- how you handle deletes (optional: is_deleted)

5.9 Lesson 9 Data Modeling: Choose Grain + Build Star Schema

Scenario: Analytics wants dashboards.

You will decide:

- grain of fact table (one row per order line OR order)

You will produce:

- dim_customer, dim_product, fact_orders

Must include in design doc

- grain statement
- primary keys
- slowly changing dimension choice (SCD1 vs SCD2)
- why you chose it

5.10 Lesson 10 dbt Core Take-Home: Staging → Marts

Task:

- create stg_orders, stg_customers
- create mart_daily_revenue
- add 4 tests:
 - unique
 - not_null
 - accepted_values
 - relationships

Must include

- dbt docs description (even if just YAML docs)

5.11 Lesson 11 Performance Basics: Make a Slow Query Fast

Scenario: You are given a slow SQL query.

Task:

- rewrite query
- propose indexing/partitioning strategy
- explain why it reduces scan

Deliverable

- sql/02_perf_fix.sql
- README “before vs after reasoning” (simple)

Advanced Track (Lesson 12 to 15)

These teach real interviews: streaming thinking, debugging, system design.

5.12 Lesson 12 Streaming Aggregation (Concept + Minimal Code)

Scenario: Events come from Kafka: order_created, order_paid, order_cancelled.

Task:

- compute revenue per 5 minutes
- handle late events conceptually

You will explain

- at least once vs exactly once
- watermark idea (late event window)
- dedup strategy with event_id

5.13 Lesson 13 Incident Debugging: Wrong Numbers in Dashboard

Scenario: Revenue dashboard suddenly spikes.

You will do:

- identify possible causes (duplicates, backfill, timezone shift, bad join)
- propose checks to confirm root cause
- implement one preventive validation/test

Deliverable

- root cause report in README (short, crisp)

5.14 Lesson 14 Pipeline Observability: Metrics + Logging

Task:

- log row counts per step
- log rejects count
- log runtime per step
- output a simple summary JSON

Why it matters

Interviewers want proof you can support pipelines in production.

5.15 Lesson 15 Mini System Design: End-to-End Data Platform for This Use Case

Scenario: You design a pipeline for orders + customer + product.

You will cover

- ingestion (batch + optional streaming)
- storage (lake/lakehouse/warehouse)
- transformations (dbt/spark)
- serving layer (BI/ML)
- governance (PII, access, audit)
- monitoring (freshness, SLA, alerts)

Deliverable

- one page architecture write-up (bullet points)
- one diagram description (even text is fine)

5.16 How you will choose the right 3 lessons (based on your level)

If you are **0–3 yrs**, start with:

- Lesson 1, 2, 5

If you are **3–8 yrs**, start with:

- Lesson 7, 8, 10

If you want **product company readiness**, do:

- Lesson 8, 12, 13

6.1 The 12 edge cases that break most take-homes

1) Empty input

- file exists but has 0 rows

What you do: produce empty output with correct schema, and log “0 rows processed”.

2) Missing required columns

- column not present (order_id missing)

What you do: fail-fast with clear error message (best) OR reject whole file.

3) Extra columns (schema evolution)

- new column appears unexpectedly

What you do: ignore extra columns or pass-through (but do not fail).

4) Null keys

- primary keys are null (order_id, customer_id)

What you do: reject those rows + count them.

5) Duplicate keys

- same key repeated in same file or across files

What you do: dedup by rule (keep latest by timestamp) OR merge/upsert.

6) Bad types

- amount is “10\$”, timestamp is “yesterday”

What you do: cast safely; if cast fails → reject row with reason.

7) Negative / impossible values

- negative revenue, future timestamp far ahead

What you do: reject or clamp only if business allows (usually reject).

8) Timezone / date boundary issues

- 2026-02-14 00:05 shifts partition due to timezone

What you do: standardize timezone (UTC or a chosen zone) and document it.

9) Late arriving data

- yesterday's partition gets records today

What you do: lookback window OR upsert merge strategy.

10) Out-of-order events (streaming)

- paid event arrives before created event

What you do: stateful logic or reconciliation rule, and document behavior.

11) Join explosion

- join keys are not unique, causing duplicates

What you do: enforce uniqueness before join or aggregate/dedup after join.

12) Partial rerun / double processing

- you run the same day twice

What you do: overwrite partition/day OR idempotent merge. Never append blindly.

6.2 Your “Minimum Edge-Case Coverage” rule

In every take-home, you will cover at least these 5:

1. empty input
2. missing columns
3. null keys
4. duplicates
5. type casting failures

If you do only this, you already beat most submissions.

6.3 How you show edge cases in your README (simple pattern)

Add a section:

Edge cases handled

- Empty input → outputs empty dataset, logs 0 rows
- Missing required columns → pipeline fails fast with message
- Null keys → moved to rejects with reason “missing_key”
- Duplicate order_id → keep latest by order_ts
- Type casting errors → moved to rejects with reason “invalid_type”

That’s enough. No essays.

6.4 Quick self-check before you submit (1 minute)

Ask yourself:

- “If input is empty, will my code crash?”
- “If column is missing, will it silently produce wrong output?”
- “If I run twice, will output double?”
- “If duplicates exist, will I overcount revenue?”
- “If timestamp format changes, will I fail clearly?”

If any answer is “yes”, fix that before submission.

6.5 Mini exercise (you will do this)

Pick any one lesson from Section 5 (example Lesson 1: Clean Orders).

Write **only one sentence**:

“My pipeline handles duplicates by _____.”

7.1 What interviewers expect from your tests

They expect you to test:

- **core transformation logic**
- **one or two edge cases**
- **a failure path** (rejects or fail-fast)

If you do this, your submission looks production-minded.

7.2 The 4 test types you will use (and reuse everywhere)

1) Happy path test

You prove the main output is correct for a clean input.

Example checks:

- correct row count
- correct computed revenue
- correct partition date

2) Duplicate handling test (mandatory in most data tasks)

You prove duplicates do not inflate metrics.

Example:

- input has 2 rows with same order_id
- output keeps 1 row (latest)

3) Null / missing key test

You prove you don't allow garbage keys.

Example:

- row with order_id = null
- it goes to rejects OR pipeline fails with clear error

4) Bad type / parse failure test

You prove bad values don't silently corrupt results.

Example:

- amount = "10\$" or order_ts = "abc"
- row rejected with invalid_type reason

7.3 Minimum test set by time budget

If you have only 2 hours

Write exactly **2 tests**:

1. happy path
2. duplicate handling OR null key (choose the one most risky)

If you have 4 hours

Write **3–5 tests**:

- happy path
- duplicates
- null key
- type parse fail
- empty input (optional)

If you have 8 hours

Write **5–8 tests**:

Add:

- join explosion check (if joins exist)
- rerun/idempotency behavior (if incremental)

7.4 The “Golden Dataset” trick (fastest way to test)

You keep a tiny dataset that you fully understand:

- sample_input.csv (10–30 rows)
- expected_output.csv (what the output must be)

Your test simply runs pipeline and compares output to expected.

This is high impact because reviewers trust it.

7.5 What NOT to do in tests (common mistakes)

Avoid:

- tests that only check “file exists”
- tests that don’t assert results
- random data without expected output
- huge datasets in tests

Your tests should be small, readable, and deterministic.

7.6 How you explain tests in README (simple)

Add a section:

Tests included

- `test_happy_path`: verifies daily revenue matches expected output
- `test_dedup_latest`: verifies duplicate `order_id` keeps latest by timestamp
- `test_null_key_reject`: verifies null `order_id` goes to rejects

That's enough.

7.7 Mini exercise (quick)

Write one line you will use in README:

“I added tests to cover happy path + duplicate handling + null key rejection, because these are the most common hidden failures in take-homes.”

8.1 The 5 validation categories you will always think about

1) Schema validations

You check:

- required columns exist
- datatypes are reasonable

Example rules

- `order_id` must exist
- `order_ts` must be parseable datetime
- amount must be numeric

Failure action

- missing required column → **fail-fast**
- wrong type → **reject row** (if possible)

2) Completeness validations

You check:

- required fields are not null

Example rules

- order_id not null
- customer_id not null

Failure action

- if key is null → **reject row** (or fail-fast if key is mandatory)

3) Uniqueness validations

You check:

- keys are unique (or you enforce a dedup rule)

Example

- order_id should be unique in clean output

Failure action

- dedup with rule and log how many were removed

4) Range + domain validations

You check:

- values fall within expected ranges or accepted categories

Example

- amount ≥ 0
- status in {created, paid, cancelled}

Failure action

- reject row with reason invalid_value

5) Freshness validations (simple but powerful)

You check:

- “Is this data recent enough?”

Example

- latest order_ts should be within last X days (configurable)

Failure action

- raise warning or fail (depends on problem)

8.2 The “Fail Fast vs Quarantine” decision (simple rule)

You will fail-fast when:

- required columns are missing
- output would be meaningless without the field
- schema is completely broken

You will quarantine (reject rows) when:

- only some rows are bad
- you can still produce correct output for the rest

This is a strong production signal.

8.3 The Rejects pattern (the one interviewers love)

If you quarantine bad rows, your rejects dataset must have:

reject columns

- original row data (or key fields)
- reject_reason
- reject_step (optional)
- rejected_at timestamp (optional)

Example reject reasons

- missing_key
- invalid_type_amount
- invalid_timestamp
- invalid_status
- negative_amount

And your README should say:

- “Rejected rows are saved separately with reasons so you can fix upstream.”

8.4 Your “Quality Summary” output (small add, big impact)

At the end of the run, print/log a simple summary:

- total input rows
- clean output rows
- rejected rows
- duplicates removed
- processing time

This single block makes your submission feel production-grade.

8.5 How you write quality rules in README (copy this format)

Data quality rules

- Required columns: order_id, customer_id, order_ts, amount
- Key fields: order_id not null
- Range rules: amount ≥ 0
- Domain rules: status $\in \{\text{created, paid, cancelled}\}$
- Dedup: keep latest by order_ts per order_id
- Failure handling: schema missing $\rightarrow$ fail-fast; row-level issues $\rightarrow$ quarantined

That’s it. Very clear.

8.6 Mini exercise (you can reuse everywhere)

You will write one sentence:

“If a row fails validation, I _____ (reject it / stop the pipeline) because _____.”

Example:

- “I reject row-level issues to avoid losing good data, but I fail-fast on missing schema because the output would be unreliable.”

9.1 The 7 performance levers you will check

1) Reduce full scans (most common issue)

If you process daily data, you should not scan all history daily.

What you will do

- process only new partitions
- use date filters / incremental logic
- use lookback window only when needed

What you will write

- “I designed this for incremental processing to avoid full scans.”

2) Partitioning (batch)

Partition your output by a natural filter key.

Best common keys

- order_date
- event_date
- region (only if queried often)

What you will write

- “Output is partitioned by order_date to speed up date-based queries.”

3) File sizing (data lake)

Too many small files = slow. One huge file = slow.

What you will mention

- compaction approach
- “target file size” concept (no need exact numbers)

What you will write

- “I would compact small files to reduce overhead and improve scan time.”

4) Join strategy (big reason of slow pipelines)

If joins are involved, you must show you thought about:

- join keys uniqueness
- skew
- avoiding join explosion

What you will write

- “I ensure join keys are unique on the dimension side to avoid row multiplication.”

5) Aggregation cost

Group-bys can explode memory.

What you will do

- pre-filter
- dedup before aggregation
- aggregate at the right grain

What you will write

- “I dedup before aggregation to avoid overcounting and extra compute.”

6) Caching/materialization (when relevant)

If repeated steps reuse the same dataset, cache or stage once.

What you will mention

- “I would materialize intermediate results when reused.”

7) Compute cost awareness (cloud mindset)

Even if local, you should show cost thinking.

What you will write

- “Incremental loads + partition pruning reduces compute cost.”

9.2 The “Performance Notes” section you will paste into README

You will add this short block:

Performance & Cost notes

- Incremental processing avoids scanning full history daily.
- Output is partitioned by order_date to enable partition pruning.
- Dedup is applied before aggregation to prevent overcount and extra compute.
- Join keys should be unique to avoid join explosion.
- If deployed at scale, I would add compaction to reduce small files.

This is enough to impress.

9.3 Common performance red flags (avoid these)

- reading entire dataset for every run without reason
- no partitioning on output
- dedup after aggregation (often wrong + expensive)
- joins without checking key uniqueness
- too many small output files

9.4 Mini exercise (one line)

For any take-home you do, you will write:

“To keep it scalable, I avoided full scans by ____.”

Example: “processing only new partitions using a checkpoint date.”

10.1 The 6 things you will always log (minimum standard)

1) Start + end of the run

- start time
- end time
- total duration

2) Input row count

- how many rows you read

3) Output row count

- how many rows you wrote

4) Reject count (if you quarantine)

- number of rejected rows
- top 3 reject reasons

5) Duplicate count (if dedup exists)

- how many duplicates removed

6) Step-level timings (simple)

- read time
- transform time
- write time

This alone makes your submission “production-ish”.

10.2 The “Run Summary” block (high impact, low effort)

At the end, you will print/log something like:

- rows_in = 1,000,000
- rows_out = 980,000
- rows_rejected = 20,000
- duplicates_removed = 5,000
- runtime_seconds = 42
- latest_processed_date = 2026-02-14

Even if your dataset is small, this pattern shows maturity.

10.3 What you will do on failure (must be clear)

You will always choose one:

Option A: Fail-fast

- stop run
- show clear error message
- do not write partial output

Option B: Partial success + rejects

- write clean output
- write rejects separately
- still report counts

But you must document it in README.

10.4 Freshness + completeness signals (simple, interview-winning)

If the task is daily processing, you will add:

- “latest timestamp processed”
- “which partitions/dates processed”

This answers the interviewer's silent question:

"How do you know the data is updated?"

10.5 How you write observability in README (copy format)

Observability

- Logs row counts at each step (read, clean, write)
- Logs rejects count + top reject reasons
- Logs duplicates removed
- Prints run summary with runtime + latest processed date

This is enough.

10.6 Common observability mistakes

Avoid:

- printing everything row by row
- no counts at all
- failures without readable error message
- writing partial output without mention

10.7 Mini exercise (one line)

You will write:

"At the end of the run, I print a summary with rows_in, rows_out, rejects, duplicates_removed, and runtime so debugging is fast."

Section 11: Documentation That Reviewers Love (README + Design Doc Templates)

If your code is good but your README is weak, reviewers won't trust you.

So you will treat documentation as part of the solution.

11.1 The README Template (copy-paste structure)

1) Title + 2-line problem restatement

You will write:

- what input is
- what output is
- what transformation you did

Example:

- “This pipeline reads daily orders CSV and produces clean, deduplicated orders partitioned by order_date.”

2) How to run (copy-paste commands)

You will include:

- install steps
- run command
- where output will be written

If the reviewer cannot run it fast, you lose points.

3) Assumptions (3–7 bullets)

You will write only what matters:

Examples:

- “Input may contain duplicates by order_id.”
- “order_ts is expected in ISO format; invalid timestamps are rejected.”
- “Reruns are possible, so output is overwritten per order_date.”

4) Edge cases handled (5 bullets)

Use the Section 6 format:

- empty input
- null keys
- duplicates
- bad types
- missing columns (fail-fast)

5) Data quality rules (short)

Use Section 8:

- required columns
- key not null
- ranges
- accepted values
- dedup rule
- failure handling rule

6) Tests included (2–6 bullets)

Mention only meaningful tests.

7) Performance & cost notes (short)

Use Section 9 template.

8) Observability (short)

Use Section 10 template.

9) Tradeoffs + improvements (this is your “senior” section)

You will write:

- what you simplified due to time
- what you would add next

Example:

- “With more time, I would add incremental checkpointing + alerts on freshness SLA.”

11.2 The 1–2 Page Design Doc Template (optional but premium)

Create docs/design.md with these headings:

1) Problem

- input, output, goal in 3 lines

2) Approach

- steps of pipeline (read → validate → transform → write)

3) Data assumptions

- keys, duplicates, timestamp rules

4) Failure handling

- fail-fast vs quarantine
- rejects format

5) Rerun & backfill strategy

- overwrite partition / merge upsert
- lookback window / checkpoint idea

6) Performance & scaling notes

- partitioning
- join risk
- file sizing / compaction idea

7) Future improvements

- CI
- better tests

- monitoring alerts
- governance/PII

Keep it short. Reviewer should finish in 2 minutes.

11.3 The “Reviewer Friendly” writing rules

You will follow these rules:

- bullets > paragraphs
- no long essays
- clear headings
- honest limitations
- no marketing language

11.4 Mini exercise (you can reuse)

Write 3 bullets for assumptions for any task:

Example:

- “Duplicates may exist for the same key.”
- “Reruns are possible, so output must be idempotent.”
- “Invalid records are quarantined with reject reasons.”

12.1 What “CI” means in take-homes

CI in take-homes = **automatic checks that prevent silly mistakes.**

If you include even basic CI, you stand out.

12.2 The only 3 checks you need

You will aim for these:

1. **Lint / formatting check** (optional but good)

2. **Run unit tests** (mandatory if you claim tests)
3. **Basic type check** (optional)

If you can't set up CI, you can still mention:

- “In production, I would add CI to run tests + lint on every PR.”

12.3 Packaging rules (so the reviewer can run it)

You will make it easy to run by adding one of these:

Option A: requirements.txt (simplest)

- list only the packages you used
- avoid unnecessary libs

Option B: pyproject.toml (cleaner, but optional)

And your README must include:

- python version (example: 3.10+)
- commands to install + run

12.4 Reproducibility (small but strong)

You will ensure:

- no absolute local file paths in code
- config-driven input/output paths
- sample input included only if allowed

This removes reviewer friction.

12.5 If you add Docker (only when asked)

Docker is not mandatory. It's a bonus only if:

- the problem explicitly asks for it
- dependencies are complex

If you add Docker, keep it minimal:

- one Dockerfile
- one run command in README

If you don't add Docker, do not apologize. Just run locally cleanly.

12.6 The “CI/CD” line you can add in README (simple)

CI/CD

- “In production, I would add a CI pipeline to run unit tests and lint checks on every PR to prevent regressions.”

If you included a GitHub Actions workflow, you can say:

- “CI runs tests automatically on push.”

12.7 Mini exercise (one sentence)

You will write:

“To avoid regressions, I would run tests automatically in CI on every change.”

13.1 Secrets handling (the golden rule)

You will never hardcode secrets inside code or README.

Examples of secrets:

- database passwords
- API keys
- tokens
- connection strings

What you will do instead

- use environment variables
- or use a config file but exclude it from git (mention in README)

✅ Strong README line:

- “Secrets are injected via environment variables and are not committed to the repo.”

13.2 PII handling (basic but powerful)

PII examples:

- email, phone, address, name, customer identifiers

If your dataset includes PII, you will do one of these

- mask sensitive columns in outputs (example: hash email)
- avoid writing raw PII in logs
- restrict sample data (use fake data if needed)

✅ Strong README line:

- “PII fields are not logged; sensitive columns can be masked before writing to output.”

13.3 Access control (conceptual mention is enough)

You will mention:

- “In production, access is controlled using RBAC / IAM roles.”
- “Only least-privilege access is provided.”

This shows you think beyond coding.

13.4 Data governance (what you will mention, short)

You will mention one or two:

- schema validation (data contract idea)
- lineage (tools like dbt docs / catalog)
- auditing (who ran what, when)

✅ Strong line:

- “I keep a run summary (rows in/out, rejects, runtime) for auditability.”

13.5 What NOT to do (major red flags)

Avoid:

- printing credentials in logs
- committing .env files
- using real customer data in sample files
- logging full rows with emails/phones

13.6 Mini exercise (one line)

You will write:

“I avoid hardcoding secrets and do not log PII to keep the pipeline safe.”

14.1 The top red flags (most common)

1) No clear “How to run”

If the reviewer struggles to run, they won’t spend time.

Fix: README run steps must be copy-paste.

2) Overengineering

If you build:

- too many layers
- too much boilerplate
- fancy patterns without need

Reviewer thinks:

“This person doesn’t know how to prioritize.”

Fix: simple structure, finish strong.

3) Silent data loss

If you drop bad rows without telling, it's a big trust issue.

Fix: quarantine rejects OR fail-fast, and document it.

4) No rerun safety (idempotency missing)

If rerunning duplicates output, it's a production disaster.

Fix: overwrite partition / merge upsert / deterministic output.

5) Duplicates not handled

Duplicates are the #1 hidden test.

Fix: dedup rule + test + count logged.

6) Join explosion risk not addressed

If you join without checking uniqueness, metrics inflate.

Fix: ensure one side is unique or dedup before join.

7) No tests (or useless tests)

If there are no tests, or tests don't assert logic, it reduces trust.

Fix: 2–5 meaningful tests only.

8) No assumptions / tradeoffs

Reviewer thinks:

“This person coded blindly.”

Fix: write assumptions + tradeoffs clearly.

9) Hardcoded paths / local machine dependency

If your code only works on your laptop, you lose points.

Fix: config-driven paths, relative paths, clear instructions.

10) Poor error messages

If a failure happens and the error message is unclear, it feels immature.

Fix: fail with readable message like:

- “Missing required column: order_id”

14.2 The “professional smell test”

Before submission, open your repo and ask:

- Can I understand the project in 30 seconds?
- Can I run it in 60 seconds?
- Do I see edge cases handled?
- Is rerun safety mentioned?
- Do tests prove key behavior?

If yes, you’re safe.

14.3 Mini exercise (simple)

Write one sentence you will include in README:

“This pipeline is rerun-safe because ____.”

Example: “it overwrites output per order_date partition.”

15.1 Pre-submit checklist (must pass all)

A) Run-ability

- README has **copy-paste run steps**

- No absolute local paths in code
- requirements.txt / environment info is present
- Output path is clear and created (or documented)

B) Correctness

- Output matches expected sample
- Metrics/aggregations are correct (spot-check 2–3 rows manually)
- No duplicate inflation in aggregates

C) Edge cases (minimum 5)

- Empty input handled
- Missing required columns fail-fast (clear message)
- Null key rows handled (reject/fail)
- Duplicate key handling implemented
- Type parse failures handled (reject/fail)

D) Data quality

- Quality rules documented (required fields, ranges, accepted values)
- Rejects strategy is clear (if used)
- Reject reasons are captured (if used)

E) Rerun safety

- You can run twice without double-counting
- You explicitly mention idempotency approach in README

F) Tests (small but meaningful)

- At least 2 tests exist
- Tests assert output logic (not just “file exists”)
- One test covers duplicates OR null keys

G) Performance & cost notes

- You mention incremental thinking / partitioning

- You mention join explosion risk if joins exist
- You mention compaction/file sizing concept (optional)

H) Observability

- Row counts logged (in/out/rejects)
- Runtime logged
- Run summary printed

I) Security basics

- No secrets committed
- No PII printed in logs (if present)

15.2 The “1-page reviewer summary” (high impact)

At the top of README (or at the end), you will add:

Reviewer Summary

- What it does:
- How to run:
- Key decisions: (dedup rule, rerun approach, validation strategy)
- Tests: (2–5 bullets)
- If I had more time: (2 bullets)

This makes review effortless.

Appendices (Ready-to-use Templates)

Appendix A: README Template (copy as-is)

Use this exact structure:

- Title
- Problem restatement (2–3 lines)
- How to run

- Input/Output schema
- Assumptions
- Edge cases handled
- Data quality rules
- Tests included
- Performance & cost notes
- Observability
- Tradeoffs + future improvements

Appendix B: Design Doc Template (1–2 pages)

Headings:

- Problem
- Approach
- Assumptions
- Failure handling
- Rerun/backfill strategy
- Performance notes
- Future improvements

Appendix C: Self-scoring rubric (score yourself before you submit)

You will rate yourself out of 10 for each:

- Correctness (0–10)
- Edge cases (0–10)
- Code clarity (0–10)
- Tests (0–10)
- Rerun safety (0–10)

- Documentation (0–10)

If any score is below 6, you fix it before submitting.

Appendix D: “Final Notes” templates (short, medium)

Short version (3 bullets)

- Key decisions:
- Known limitations:
- Next improvements:

Medium version (6 bullets)

- Why this approach
- Dedup rule
- Rerun strategy
- Validation strategy
- Test coverage
- Scale improvements

16.1 Scenario: Batch Orders Pipeline (Messy CSV → Clean Parquet)

Setup: You receive daily orders.csv. You must clean it and write partitioned output.

Q1) What is your approach end-to-end?

Your answer:

You start by restating the problem: you will read daily orders, validate schema, clean types, deduplicate by order_id, quarantine bad rows, and write clean output partitioned by order_date. You will keep it rerun-safe by overwriting the same date partition. You will log row counts (input/output/rejects/duplicates removed) so you can debug quickly.

Q2) How do you handle duplicates?

Your answer:

You treat duplicates as a default assumption. You pick a clear rule: “keep the latest record

per order_id using order_ts (or updated_at)”. You log how many duplicates you removed. You also add a test that proves duplicates do not inflate counts or revenue.

Q3) What do you do with invalid rows (null keys, bad timestamps, negative amount)?

Your answer:

You do not drop rows silently. If order_id is missing, you quarantine that row into a rejects dataset with reject_reason = missing_key. If order_ts parsing fails or amount cannot be cast, you quarantine with invalid_type or invalid_timestamp. If a required column is missing from the file, you fail-fast because output becomes unreliable.

Q4) How do you make this rerun-safe?

Your answer:

You avoid append-only writes for the same partition. You write output partitioned by order_date, and if you rerun the same day, you overwrite that partition (or write to a temp path and swap). That way, running the pipeline twice does not double-count.

Q5) What tests do you write?

Your answer:

You write small, high-signal tests:

- Happy path test: verifies clean output matches expected results.
 - Dedup test: same order_id appears twice → only latest is kept.
 - Null key test: null order_id goes to rejects.
- If time allows, you add a “bad timestamp” test and an “empty file” test.

16.2 Scenario: Incremental + CDC Upsert (Late Updates)

Setup: Orders can be updated late (status changes). You must maintain latest state.

Q1) How do you design incremental processing?

Your answer:

You use a checkpoint approach: you track last processed date/time (or watermark), and process only new data. Because updates can arrive late, you also keep a configurable lookback window (example: reprocess last 3 days). You then merge/upsert into the target using order_id as the key and updated_at for latest selection.

Q2) How do you implement upsert logic in a take-home (even without real warehouse)?

Your answer:

You explain it clearly: you read the existing target snapshot (or prior output), union it with incoming delta, and keep the latest row per order_id by updated_at. If the platform supports merges (Snowflake/Delta), you mention you would use MERGE for correctness and performance.

Q3) How do you avoid overcounting when updates arrive?**Your answer:**

You avoid treating updates as new rows in analytics tables. You separate “latest state” table (fact snapshot) from “event log”. For metrics like revenue, you either calculate from the event-level paid transactions or ensure your snapshot table is not used for additive metrics unless designed for it.

Q4) What are the biggest edge cases?**Your answer:**

- Update arrives without original create record (out-of-order)
 - Two updates have same updated_at (tie-breaker needed)
 - Partial file reruns
- You handle ties using a deterministic rule (example: pick latest ingestion time or stable ordering). You keep rerun safety via merge logic.

16.3 Scenario: Streaming Aggregation (Late Events + Dedup)

Setup: Kafka-like events: order_created, order_paid, order_cancelled. You compute revenue per 5 minutes.

Q1) What delivery semantics do you assume?**Your answer:**

You assume at-least-once delivery is possible, which means duplicates can happen. So you implement dedup using a stable event key (event_id) or (order_id, event_type, event_ts) depending on available fields. You never assume “no duplicates” in streaming.

Q2) How do you handle late events?**Your answer:**

You use watermarking with an allowed lateness window (example: accept events up to 10 minutes late). Within that window you update aggregates. Beyond it, you either drop, route

to a late-events stream, or trigger correction logic depending on requirements. You write this clearly in your design notes.

Q3) How do you ensure correctness in aggregations?

Your answer:

You keep state per window and update it based on event time, not processing time. You also ensure dedup happens before aggregation so revenue isn't inflated.

Q4) How do you explain checkpointing?

Your answer:

You say you checkpoint offsets/state so if the job restarts, it resumes without losing progress. You mention you monitor lag and late-event counts as key signals.

16.4 Scenario: dbt Modelling + Metric Standardization

Setup: You build dbt models to standardize business metrics.

Q1) How do you structure dbt models?

Your answer:

You follow layers: stg_* for cleaned raw tables, int_* for joins/logic reuse, and mart_* for final business outputs. You keep transformations transparent and testable.

Q2) How do you ensure metrics are consistent?

Your answer:

You define metric logic in one place (mart or metric layer) and reuse it. You avoid duplicating "revenue" logic across multiple models. You document definitions clearly, so dashboards and ML use the same truth.

Q3) What tests do you add?

Your answer:

You add:

- unique + not_null for primary keys
- relationships for foreign keys
- accepted_values for status

You also add freshness tests (if supported) or at least mention freshness checks in orchestration.

Q4) How do you handle slowly changing dimensions?

Your answer:

You choose SCD type based on the business question. If history matters (address changes), you use SCD2. If only latest matters, SCD1 is enough. You state the grain clearly.

16.5 Scenario: Data Quality Incident (Revenue Spike)

Setup: Dashboard revenue spikes suddenly. You must debug.

Q1) What are your first checks?

Your answer:

You start with quick sanity checks:

- Did input volume spike?
- Did duplicates increase?
- Did join row counts explode?
- Did a backfill rerun append instead of overwrite?

You compare row counts and revenue totals day-over-day and isolate where the change began.

Q2) How do you find root cause quickly?

Your answer:

You break the pipeline into steps and compare counts per step (read → cleaned → joined → aggregated). If the jump happens after a join, you check key uniqueness. If the jump happens at ingestion, you check duplicates and rerun behavior.

Q3) What permanent fix do you add?

Your answer:

You add a validation guardrail: if revenue deviates beyond a threshold (example: 3x), the pipeline flags it (or stops) and logs top drivers. You also add a test for join uniqueness or dedup logic to prevent recurrence.

16.6 Scenario: Performance Tuning (Skew + Small Files)

Setup: Your job is slow, spills, or timeouts.

Q1) What is your tuning approach?

Your answer:

You first measure where time goes (read vs join vs write). You then check skew (one key dominates), file sizes (too many small files), and join strategy (broadcast vs shuffle). You avoid guessing—your decision is driven by observed counts and distributions.

Q2) How do you handle skew?

Your answer:

You identify skewed keys and choose a mitigation: salting keys for heavy hitters, pre-aggregating before join, or handling heavy keys separately. You also ensure dimension tables are deduped and unique by join key.

Q3) How do you handle small files?

Your answer:

You compact outputs and control the number of files per partition. You also avoid writing too many partitions with tiny volumes. You mention this as both performance and cost improvement.

16.7 Scenario: Backfill + Rerun Safety (Date Range Processing)

Setup: Data is wrong for last week; you must backfill safely.

Q1) How do you design backfill mode?

Your answer:

You add config/CLI parameters `start_date` and `end_date`. You process per date partition in a loop. For each date, you overwrite only that partition (not the full table). That makes it safe and contained.

Q2) What is your idempotency guarantee?

Your answer:

You guarantee that rerunning the same date produces the same output by deterministic transformations + overwrite/merge strategy. You also log “dates processed” and “partitions overwritten”.

Q3) What common mistake do you avoid?

Your answer:

You avoid mixing append and overwrite for the same partition. That is the biggest source of duplicate totals after backfills.

16.8 Scenario: Joins + Join Explosion Prevention

Setup: You join orders with customers and metrics inflate.

Q1) Why does join explosion happen?

Your answer:

It happens when your join key is not unique on one or both sides. One order joins multiple customer records and multiplies rows. Revenue inflates without any obvious error.

Q2) How do you prevent it?

Your answer:

You enforce uniqueness on the dimension side before join (dedup by latest updated_at). You validate uniqueness with a check (count distinct key vs total). If not unique, you fail-fast or quarantine duplicates depending on needs.

Q3) What test do you add?

Your answer:

You add a test that asserts dimension keys are unique before join. This prevents silent metric inflation.

16.9 Scenario: Security/PII + Secrets (Real-World Safe Handling)

Setup: Data contains emails/phones; you must handle responsibly.

Q1) What do you do with secrets?

Your answer:

You never hardcode them. You inject secrets via environment variables or secret managers in real deployments. In a take-home, you keep placeholders and clearly document how to pass credentials securely.

Q2) What do you do with PII?

Your answer:

You do not log PII. If you must store it, you mask or hash where possible. You also restrict

sample data (fake data) in the repo. You mention RBAC/least privilege as the access control principle.

16.10 Scenario: Final Review Round (Tradeoffs + Improvements)

Setup: Interviewer asks you to explain decisions and what you'd improve.

Q1) Why did you choose this approach?

Your answer:

You chose the simplest approach that guarantees correctness, rerun safety, and debuggability. You did not overengineer. You focused on what the reviewer grades: correctness + edge cases + tests + clear documentation.

Q2) What tradeoffs did you make?

Your answer:

You name realistic tradeoffs:

- you kept validations lightweight instead of building a full framework
 - you used overwrite-per-partition instead of complex incremental merge (if not required)
 - you prioritized high-signal tests over large test suites
- This shows maturity and prioritization.

Q3) What would you add with more time?

Your answer:

You add improvements that match real production:

- CI to run tests on every change
- data freshness alerts / anomaly detection
- better incremental checkpointing
- stronger schema evolution handling
- compaction strategy + performance monitoring

You keep it short and concrete.

Top 10 resources

Step 1) Build 1 complete end-to-end pipeline (best foundation)

- Use this as your “hands-on track” for learners (fits Section 0 + Scenario 16.1/16.7)
- GitHub (full curriculum): <https://github.com/DataTalksClub/data-engineering-zoomcamp>
- Course page (quick navigation): <https://datatalks.club/docs/courses/data-engineering-zoomcamp/>

Step 2) Add real data validation (Rejects/Quarantine pattern)

- Use this under Section 8 + Scenario 16.5
- Great Expectations Checkpoints (official): <https://docs.greatexpectations.io/docs/0.18/reference/learn/terms/checkpoint/>
- Create checkpoint with actions (practical): https://docs.greatexpectations.io/docs/core/trigger_actions_based_on_results/create_a_checkpoint_with_actions/

Step 3) Add dbt tests (the interview-friendly “DQ in SQL” story)

- Use this under Section 7 + Scenario 16.4
- dbt data tests (official): <https://docs.getdbt.com/docs/build/data-tests>
- data-tests reference (properties): <https://docs.getdbt.com/reference/resource-properties/data-tests>

Step 4) Add dbt incremental models (the cleanest incremental explanation)

- Use this under Scenario 16.2
- dbt incremental models (official): <https://docs.getdbt.com/docs/build/incremental-models>

Step 5) Add dbt custom tests (so you can teach “write your own rule”)

- Use this under Section 8 + Scenario 16.4

- dbt custom generic tests (official): <https://docs.getdbt.com/best-practices/writing-custom-generic-tests>

Step 6) Teach CDC/Upsert using Delta MERGE (Lakehouse track)

- Use this under Scenario 16.2

- Delta MERGE upsert (Databricks official): <https://docs.databricks.com/aws/en/delta/merge>

Step 7) Teach CDC/Upsert using Snowflake MERGE (Warehouse track)

- Use this under Scenario 16.2

- Snowflake MERGE (official): <https://docs.snowflake.com/en/sql-reference/sql/merge>

Step 8) Teach streaming delivery semantics (avoid the “exactly once” confusion)

- Use this under Scenario 16.3

- Confluent Kafka delivery semantics: <https://docs.confluent.io/kafka/design/delivery-semantics.html>

Step 9) Teach watermarking + late events correctly (Spark Structured Streaming)

- Use this under Scenario 16.3

- Spark Structured Streaming guide (official): <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Step 10) Teach performance + CI basics (the “production-ready” signals)

- Use this under Section 9 + Section 12 + Scenario 16.6

- Spark SQL performance tuning (official): <https://spark.apache.org/docs/latest/sql-performance-tuning.html>

- GitHub Actions (Python CI): <https://docs.github.com/actions/guides/building-and-testing-python>
- pytest getting started: <https://docs.pytest.org/en/stable/getting-started.html>
- pyproject.toml guide (packaging): <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>

DataGeeks